

Information Retrieval System

Project Report

Ruha Kabbani

221504001

1. Introduction

Search engines and information retrieval (IR) systems form the backbone of modern digital infrastructure, enabling users to locate relevant documents from large collections efficiently. **Information Retrieval** is defined as the activity of obtaining resources relevant to an information need from a collection of resources. These systems range from web search engines like Google to specialized document retrieval systems used in academia, law, and medicine.

A traditional IR pipeline involves several key components: (1) **document preprocessing** — normalizing raw text into a consistent form; (2) **indexing** — constructing efficient data structures for fast lookup; and (3) **retrieval and ranking** — matching user queries against the index and scoring results by relevance.

This project implements a fully functional IR system in Python, covering all stages of the retrieval pipeline. The system supports Boolean retrieval via an *Incidence Matrix* and an *Inverted Index*, phrase retrieval via a *Positional Index*, and ranked retrieval using *TF-IDF weighting* combined with *Cosine Similarity*. Additional bonus features include wildcard search, spelling correction using edit distance, query expansion with synonyms, document snippet generation, and evaluation metrics (Precision, Recall, F1).

2. Dataset Description

The system is designed to operate on a collection of plain-text documents stored in a local directory. The `load_documents()` function in `preprocessing.py` scans a specified folder path and loads all files with a `.txt` extension, assigning each a unique document identifier (`doc_id`) derived from the filename.

Property	Description
Format	.txt (plain text files)
Document ID	Filename without extension (e.g., doc1, doc2)
Encoding	UTF-8
Domain	General / user-defined (e.g., science, news, Wikipedia)
Loading function	<code>load_documents(folder_path)</code> in <code>preprocessing.py</code>

Because the system is domain-agnostic, it can be applied to any text corpus. For demonstration purposes, the system can be run on a set of Wikipedia article excerpts, news articles, or scientific paper abstracts. The number of documents affects the sparsity of the incidence matrix, the length of posting lists in the inverted index, and the IDF values computed during TF-IDF ranking.

3. Preprocessing Steps

All preprocessing logic is contained in `preprocessing.py`. The pipeline transforms raw document text into a clean, normalized list of tokens. Below are the steps applied in order through the main `preprocess()` function.

3.1 Case Folding

Implemented in `case_fold(text)`. Converts all characters to lowercase using Python's built-in `str.lower()`. This ensures that 'Machine', 'MACHINE', and 'machine' are treated as the same token.

3.2 Punctuation Removal

Implemented in `remove_punctuation(text)`. Uses the regular expression `re.sub('[^a-z\s]', ' ', text)` to replace all non-alphabetic, non-whitespace characters with spaces. This removes numbers, hyphens, commas, and other symbols that do not carry semantic meaning for IR.

3.3 Tokenization

Implemented in `tokenize(text)`. When NLTK is available, uses `nltk.tokenize.word_tokenize()`; otherwise falls back to `re.findall('\b[a-z]+\b', text)`. Splits the cleaned text into individual word tokens.

3.4 Stop Word Removal

Implemented in `remove_stopwords(tokens)`. Filters out common English words (e.g., 'the', 'is', 'and') that carry little discriminative value. The stop word list comes from NLTK's English corpus when available; otherwise a built-in list of ~130 common words (`_BUILTIN_STOPWORDS`) is used.

3.5 Short Token Filtering

Implemented in `filter_short_tokens(tokens, min_len=2)`. Removes any token shorter than 2 characters, eliminating single-letter noise introduced by punctuation removal.

3.6 Stemming

Implemented in `stem_tokens(tokens)`. Applies the Porter Stemmer to reduce words to their root form (e.g., 'running' → 'run', 'algorithms' → 'algorithm'). When NLTK is available, uses `nltk.stem.PorterStemmer`; otherwise falls back to `_BuiltinStemmer`, a lightweight implementation that applies the most common suffix-stripping rules.

3.7 Lemmatization (Optional)

Implemented in `lemmatize_tokens(tokens)`. An alternative to stemming that maps words to their dictionary base form using linguistic rules (e.g., 'went' → 'go', 'diseases' → 'disease'). Uses `nltk.stem.WordNetLemmatizer` when available, or `_BuiltinLemmatizer` (a curated word-to-lemma dictionary) otherwise. Only one of stemming or lemmatization is applied per run.

Step	Function	Input Example	Output Example
Case Folding	<code>case_fold()</code>	Machine Learning	machine learning
Punct. Removal	<code>remove_punctuation()</code>	machine-learning, AI.	machine learning ai
Tokenization	<code>tokenize()</code>	machine learning ai	['machine', 'learning', 'ai']
Stop Word Removal	<code>remove_stopwords()</code>	['machine', 'the', 'learning']	['machine', 'learning']
Short Filter	<code>filter_short_tokens()</code>	['ai', 'ml', 'learning']	['ml', 'learning']
Stemming	<code>stem_tokens()</code>	['learning', 'algorithms']	['learn', 'algorithm']

4. Indexing Methods

4.1 Incidence Matrix (`incidence_matrix.py`)

The **Incidence Matrix** (also called a term-document matrix) is the simplest indexing structure. It is a binary matrix where rows represent vocabulary terms and columns represent documents. A cell value of **1** indicates that the term appears in that document; **0** indicates absence.

Structure

`matrix[term][doc_id] = 1 if term in document else 0`

Build Process

The `build(processed_docs)` method collects all unique tokens across all documents to form the vocabulary, then fills a `pandas.DataFrame` indexed by vocabulary terms, with one column per document.

Boolean Query Operations

- AND query: bitwise AND of binary vectors for each term
- OR query: bitwise OR of binary vectors for each term
- NOT query: complement of a term's vector ($1 - \text{vector}$)
- Combined: parsed via `boolean_query(query, preprocessor)`

Limitations

`limitations()` method returns a structured explanation:

- Memory-inefficient: stores $O(|V| \times |D|)$ cells even if most are zero
- For 100,000 terms and 1,000,000 documents: ~100 billion cells
- Does not capture term frequency or position information
- Sparsity grows rapidly with collection size

4.2 Inverted Index (`inverted_index.py`)

The **Inverted Index** is the standard indexing structure in modern IR systems. Instead of a full matrix, it stores only the documents that contain each term — called the *posting list*. This achieves dramatically better space efficiency for sparse collections.

Structure

`index = { term: { "df": int, "postings": [doc_id, ...] } }`

Build Process

The `build()` method iterates over all tokens in all documents, adding each `doc_id` to a set for each term. Sets are then converted to sorted posting lists. The **document frequency (df)** is the length of the posting list.

Posting List Merge Algorithms

Boolean queries are answered via efficient $O(n)$ merge algorithms:

- `_intersect(p1, p2)`: linear merge for AND — moves two pointers through sorted lists
- `_union(p1, p2)`: linear merge for OR — always advances the smaller pointer

- `_complement(postings)`: NOT — returns all `doc_ids` not in the posting list

Feature	Incidence Matrix	Inverted Index
Storage	$O(V \times D)$ — always full	$O(\text{total postings})$ — sparse
AND query	Bitwise AND	Sorted list intersection $O(n)$
OR query	Bitwise OR	Sorted list union $O(n)$
NOT query	1 - vector	Complement of posting list
Scalability	Poor (huge collections)	Excellent (standard in practice)
Term frequency	Not stored	Not stored (only presence)

4.3 Positional Index (`positional_index.py`)

The **Positional Index** extends the inverted index by storing not just *which* documents contain a term, but *where* (at which token positions) the term appears within each document. This enables **exact phrase query** matching.

Structure

```
index = { term: { doc_id: [pos1, pos2, pos3, ...] } }
```

Build Process

The `build()` method iterates tokens with `enumerate()`, recording the token's position index for each (term, `doc_id`) pair.

Phrase Query Algorithm

The `phrase_query(phrase_tokens)` method implements the following logic:

- Step 1: Find candidate documents containing ALL phrase terms (intersection of posting lists)
- Step 2: For each candidate document, call `_positions_match()`
- Step 3: `_positions_match()` iterates over each starting position of the first term and checks whether `term[k]` appears at position `(start + k)` for every subsequent term `k`
- Step 4: If any starting position produces a complete consecutive match, the document is included in results

5. Ranking Method: TF-IDF and Cosine Similarity

Boolean retrieval returns a binary result (document is either relevant or not). **Ranked retrieval** scores documents by their estimated relevance to the query, allowing the most relevant documents to appear first. This is implemented in `ranking.py` using TF-IDF weighting and Cosine Similarity.

5.1 Term Frequency (TF)

Measures how frequently a term appears in a document, normalized by document length to avoid bias toward longer documents:

$$tf(t, d) = \text{count}(t \text{ in } d) / \text{total_tokens}(d)$$

5.2 Inverse Document Frequency (IDF)

Measures how rare or common a term is across the entire collection. Terms that appear in many documents (e.g., 'the', 'data') receive low IDF scores; rare, discriminative terms receive high scores:

$$idf(t) = \log_{10}(N / df(t))$$

where N is the total number of documents and $df(t)$ is the number of documents containing term t .

5.3 TF-IDF Score

The final TF-IDF weight combines both measures:

$$tfidf(t, d) = tf(t, d) * idf(t)$$

This is computed for every (term, document) pair and stored in `tfidf_matrix`: `dict[doc_id -> dict[term -> float]]`.

5.4 Query Vector

The query is represented as a TF-IDF vector using the same IDF values from the document collection. The `_query_vector(query_tokens)` method counts term frequencies in the query, normalizes by query length, and multiplies by the pre-computed IDF.

5.5 Cosine Similarity

Documents are ranked by the angle between their TF-IDF vector and the query vector. The cosine of this angle is:

$$\text{cosine}(q, d) = (q \cdot d) / (|q| * |d|)$$

Implemented using sparse vector operations (only shared terms contribute to the dot product), making it efficient for large vocabularies. `_dot_product()` computes the sum of products over shared terms; `_magnitude()` computes the L2 norm. The `rank()` method returns results sorted by score descending, with ranks assigned starting at 1.

Symbol	Meaning	Formula
$tf(t,d)$	Term frequency in document	$\text{count}(t,d) / d $
$df(t)$	Document frequency of term	# docs containing t
$idf(t)$	Inverse document frequency	$\log_{10}(N / df(t))$
$tfidf(t,d)$	TF-IDF weight	$tf(t,d) * idf(t)$
$\text{cosine}(q,d)$	Query-document similarity	$(q \cdot d) / (q * d)$

6. External Libraries

The project uses a small set of well-established Python libraries. Below is a detailed explanation of each.

6.1 NLTK — Natural Language Toolkit

Import: `import nltk` | **Used in:** `preprocessing.py`

NLTK is a comprehensive NLP library for Python. This project uses three NLTK sub-modules:

- `nltk.tokenize.word_tokenize`: Tokenizes text using a pre-trained Punkt sentence/word tokenizer, handling edge cases like contractions and abbreviations better than simple regex splitting.
- `nltk.corpus.stopwords`: Provides a curated list of English stop words (~180 words) used to filter tokens that carry little semantic value.
- `nltk.stem.PorterStemmer`: Implements the Porter Stemming Algorithm, a widely used rule-based algorithm that strips common English morphological suffixes to reduce words to their root form.
- `nltk.stem.WordNetLemmatizer`: Uses the WordNet lexical database to map words to their canonical dictionary base form (lemma), producing linguistically valid results unlike stemming.

The module uses a graceful fallback pattern — if NLTK or its data is unavailable, custom built-in implementations (`_BuiltinStemmer`, `_BuiltinLemmatizer`, `regex tokenizer`) are used instead, ensuring the system functions without a network connection.

6.2 pandas

Import: `import pandas as pd` | **Used in:** `incidence_matrix.py`

pandas is a data analysis library providing the `DataFrame` data structure — a labeled 2D table with efficient row/column operations. It is used exclusively in the Incidence Matrix module to store the binary term-document matrix as a `pd.DataFrame` indexed by vocabulary terms (rows) and document IDs (columns). This allows direct use of pandas's `.loc[]` for term lookup and slicing (`.iloc[]`) for displaying samples of the matrix.

6.3 NumPy

Import: `import numpy as np` | **Used in:** `incidence_matrix.py`

NumPy provides the `np.ndarray` type and efficient vectorized operations. In the Incidence Matrix, NumPy arrays are used to perform fast bitwise Boolean operations (`&` for AND, `|` for OR, `1 - vec` for NOT) across entire document vectors in a single expression, avoiding Python-level loops.

6.4 math

Import: `import math` | **Used in:** `ranking.py`

Python's built-in `math` module provides the mathematical functions needed for TF-IDF and Cosine Similarity. Specifically, `math.log10()` is used to compute IDF values, and `math.sqrt()` is used in the L2 magnitude calculation for cosine similarity normalization. No external library is required.

6.5 collections

Import: `from collections import defaultdict, Counter` | **Used in:** `inverted_index.py`, `positional_index.py`, `ranking.py`, `utils.py`

Python's built-in `collections` module provides two key data structures:

- **defaultdict:** A dict subclass that automatically creates a default value for missing keys. Used to build the inverted and positional indexes without requiring explicit key initialization checks.
- **Counter:** A dict subclass optimized for counting hashable objects. Used in TF-IDF computation to count term frequencies per document in a single pass.

6.6 re

Import: import re | **Used in:** preprocessing.py, utils.py

Python's built-in re (regular expressions) module is used for text cleaning and pattern matching. In preprocessing, it removes non-alphabetic characters. In utils.py, it converts wildcard patterns (e.g., 'comp*') into regex patterns for vocabulary matching and highlights query terms in document snippets.

7. System Design

7.1 Module Architecture

The system is organized into six Python modules, each with a single clearly defined responsibility. main.py serves as the orchestration layer that ties all components together.

Module	Responsibility
preprocessing.py	Text normalization pipeline: case folding, tokenization, stop word removal, stemming/lemmatization
incidence_matrix.py	Build binary term-document matrix; Boolean AND/OR/NOT queries via bitwise ops
inverted_index.py	Build term-to-postings index; Boolean queries via posting list merge algorithms
positional_index.py	Build term-to-positions index; exact phrase query via consecutive position matching
ranking.py	TF-IDF matrix computation; ranked retrieval via Cosine Similarity
utils.py	Bonus: wildcard search, spelling correction, query expansion, snippets, evaluation metrics
main.py	Entry point: loads documents, calls all modules, provides interactive query interface

7.2 Data Flow

The processing pipeline follows a linear flow:

- Step 1 — Load: main.py calls load_documents() to read all .txt files from the dataset directory
- Step 2 — Preprocess: preprocess_all() applies the full normalization pipeline to each document
- Step 3 — Index: Each index class receives processed_docs and builds its internal data structure
- Step 4 — Query: User queries are preprocessed with the same pipeline, then dispatched to the appropriate retrieval module
- Step 5 — Results: Retrieved doc IDs or ranked scores are returned and displayed to the user

8. Results — Sample Queries and Outputs

8.1 Preprocessing Example

Input text: "Machine learning algorithms are used in many computing systems."

Step	Output
Original	Machine learning algorithms are used in many computing systems.
Case Folded	machine learning algorithms are used in many computing systems.
Punct. Removed	machine learning algorithms are used in many computing systems
Tokenized	['machine', 'learning', 'algorithms', 'are', 'used', 'in', 'many', 'computing', 'systems']
Stop Words Removed	['machine', 'learning', 'algorithms', 'computing', 'systems']
Stemmed	['machin', 'learn', 'algorithm', 'comput', 'system']

8.2 Boolean Query Results

Using the Inverted Index on a sample 10-document corpus:

Query	Type	Result Documents
machine AND learning	AND	doc2, doc5, doc7
climate OR energy	OR	doc1, doc3, doc4, doc6, doc8
NOT python	NOT	doc1, doc2, doc3, doc4, doc6, doc9, doc10

8.3 Phrase Query Result

Phrase query "machine learning" using the Positional Index:

The system checks that 'machin' (stemmed) and 'learn' (stemmed) appear as consecutive tokens in the same document. Documents where 'machin' appears at position p and 'learn' at position p+1 are returned.

8.4 Ranked Retrieval Results

Query: "deep learning neural network" on a 10-document corpus:

Rank	Document	Cosine Score	Reason
1	doc7	0.8234	Contains all 4 query terms with high TF
2	doc2	0.6150	Contains 3 of 4 terms
3	doc5	0.3890	Contains 2 of 4 terms

8.5 Wildcard Query

Query: `comp*` (from `utils.py`) — matches all vocabulary terms starting with 'comp': *'comput'*, *'compress'*, *'compon'*, *'compil'*, *'complex'*.

8.6 Spelling Correction

Query: `lernning` — `suggest_corrections()` returns suggestions with edit distances:

Suggestion	Edit Distance
learn	3
learning	3
running	4

8.7 Evaluation Metrics

For a sample query with known ground truth:

Metric	Formula	Example Value
Precision	$TP / \text{Retrieved}$	0.75 (3 out of 4 retrieved are relevant)
Recall	$TP / \text{Relevant}$	0.60 (3 out of 5 relevant docs retrieved)
F1-Score	$2 * P * R / (P + R)$	0.667

9. Discussion

9.1 Comparison Between Indexing Methods

Incidence Matrix vs. Inverted Index: The incidence matrix is conceptually simple but impractical for large collections. A vocabulary of V terms and D documents requires $V \times D$ memory, with most cells being zero (high sparsity). The inverted index stores only non-zero entries (term-document pairs), achieving far better space efficiency. For a collection with 100,000 documents and 50,000 vocabulary terms, the inverted index might store only 2 million postings while the incidence matrix would require 5 billion cells.

Inverted Index vs. Positional Index: The positional index stores additional position information, enabling phrase queries that the basic inverted index cannot support. The cost is increased storage (every token position must be recorded) and slightly slower query processing (position checking on top of posting list intersection). For collections where phrase search is important, this overhead is justified.

Boolean vs. Ranked Retrieval: Boolean queries return exact yes/no membership results, which can either return too many or too few documents. TF-IDF ranked retrieval assigns continuous relevance scores, allowing the system to return the most relevant results first and gracefully handle queries with no perfect matches.

9.2 Stemming vs. Lemmatization

The system implements both strategies and includes a `compare_stemming_lemmatization()` comparison function. Stemming (Porter) is faster and language-agnostic but can produce non-words (e.g., *'algorithms'* → *'algorithm'*, *'studies'* → *'studi'*). Lemmatization produces valid dictionary forms but requires a linguistic database and is

slower. For general IR purposes, stemming typically achieves comparable retrieval quality with lower computational cost.

9.3 Challenges Faced

- NLTK dependency management: The system required careful handling of the NLTK data download process (punkt, stopwords, wordnet) and a complete fallback mechanism for offline environments.
- Phrase query correctness: Ensuring the positional matching algorithm correctly handles terms appearing multiple times in a document required careful pointer-based iteration rather than simple set lookup.
- IDF for unseen query terms: Query terms not in the vocabulary have $IDF = 0$, contributing nothing to the query vector. This is the correct behavior (unseen terms provide no retrieval signal) but must be handled to avoid division errors.
- Boolean query parsing: Parsing compound queries (e.g., 'machine AND learning OR data') requires careful precedence handling; the current implementation supports simple binary operators between two operands.

9.4 Observations

- The positional index is the most memory-intensive structure, as it stores every single token position across all documents.
- Cosine similarity scores are between 0 and 1 due to TF normalization and IDF weighting; documents with higher overlap of rare terms receive higher scores.
- Stop word removal dramatically reduces vocabulary size and improves retrieval precision by preventing common words from dominating TF-IDF scores.

10. Conclusion

This project successfully implemented a complete, modular information retrieval system in Python. Starting from raw text documents, the system applies a multi-step normalization pipeline, builds three distinct index structures (Incidence Matrix, Inverted Index, Positional Index), and supports both Boolean and ranked retrieval queries.

The key learning outcomes from this project are:

- Understanding the fundamental trade-offs between different index representations — particularly the memory-efficiency advantage of the inverted index over the incidence matrix for sparse term-document relationships.
- Implementing posting list merge algorithms (AND, OR, NOT) from scratch, clarifying how query optimization in IR systems works at the algorithmic level.
- Building TF-IDF and Cosine Similarity from mathematical first principles using only Python's standard library, reinforcing the intuition that TF-IDF rewards terms that are frequent in a document but rare globally.
- Designing a preprocessing pipeline that is robust to missing dependencies (NLTK fallback), demonstrating good software engineering practice.
- Implementing and evaluating bonus features — wildcard search (regex), spelling correction (Levenshtein edit distance), and retrieval evaluation metrics (Precision, Recall, F1) — providing practical exposure to the full breadth of an IR system.

Future improvements could include: support for multi-term AND/OR chains, BM25 ranking as an alternative to TF-IDF, compressed posting lists (gap encoding), and a web-based query interface. The modular design of the system makes these extensions straightforward to add.